

Detector de Muones de Bajo Costo Guía Completa de Construcción y Programación

ionnhatan.garcia@umce.cl

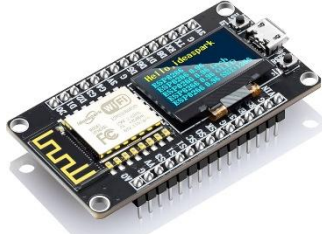
Introducción

Este tutorial describe cómo construir un detector de muones de bajo costo utilizando dos contadores Geiger-Müller comerciales tipo **CAJOE RadiationD-v1.1**, un microcontrolador **NodeMCU ESP8266** con una pantalla **OLED integrada**. El sistema implementa coincidencias temporales entre ambos detectores para identificar partículas de alta energía que atraviesan ambos tubos casi simultáneamente.

El objetivo es entregar una guía clara, secuencial y didáctica que permita a estudiantes, docentes y aficionados replicar el detector sin dificultades.

Materiales

Componente	Cantidad	Notas
NodeMCU ESP8266 con OLED integrada	1	Cualquier versión con pines D1–D8
Detector GM CAJOE RadiationD-v1.1	2	Tubo J315β integrado
Fuente 5V 1A para sensores	1	Obligatoria para evitar sobrecarga del ESP8266
Cables Dupont	10–15	
Protoboard o PCB	1	Opcional
Computador con Arduino IDE	1	Para cargar el firmware

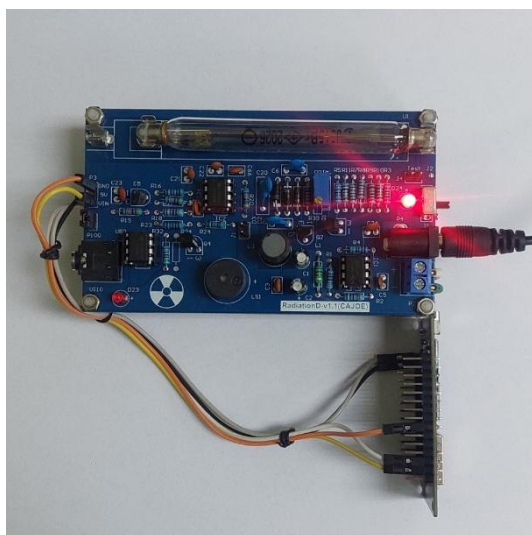
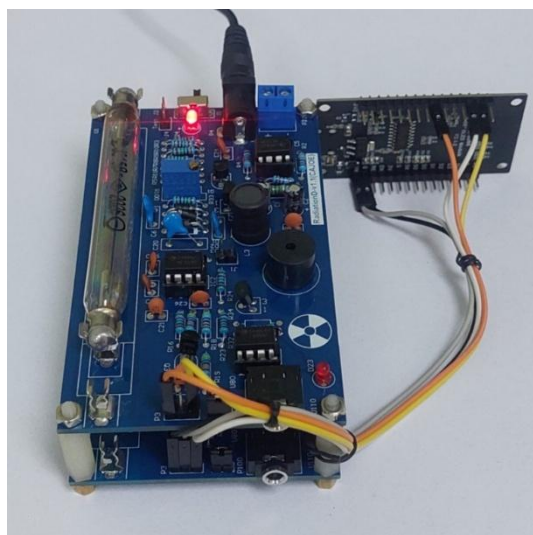
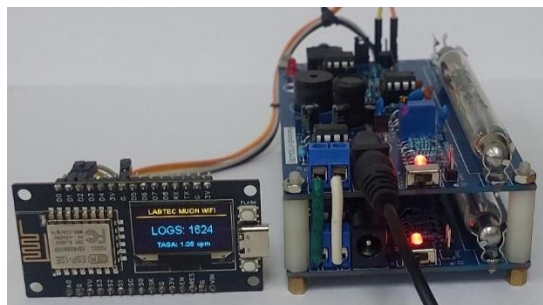
Microcontrolador NodeMCU-ESP8266 	Detector Geiger tipo CAJOE (RadiationD-v1.1) 
---	--

Principio de funcionamiento

El sistema de detección de muones se compone de dos contadores Geiger comerciales tipo CAJOE (RadiationD-v1.1), cada uno basado en un tubo cilíndrico Geiger-Müller (GM) tipo J315β con electrónica integrada de alta tensión y conformación de pulsos. Cada contador GM requiere ser conectado a una fuente de 5V para su funcionamiento. La lógica de detección por coincidencia se implementó a partir de un sistema basado en el microcontrolador NodeMCU-ESP8266 que analiza la activación de los dos detectores GM dentro de un intervalo de tiempo muy breve, almacenando temporalmente los datos en un búfer y gestionando su envío a Internet. La lógica principal se basa en el uso de interrupciones de hardware, las cuales registran el tiempo exacto en microsegundos cada vez que un sensor detecta una partícula. Al calcular la diferencia de tiempo

entre los pulsos generados en ambos detectores, el software determina si el evento fue causado por una única partícula de alta energía que atravesó ambos tubos casi simultáneamente.

Imágenes del detector armado



Conexiones eléctricas

Para que el detector de muones funcione correctamente con el microcontrolador NodeMCU-ESP8266, es necesario realizar conexiones específicas que permitan capturar los pulsos de detección y mostrar los datos en la pantalla OLED que viene integrada.

A continuación, se detallan las conexiones eléctricas basadas en el código del sensor:

1. Sensores de Radiación (Pines de Interrupción)

El detector utiliza dos sensores para identificar coincidencias. Estos deben conectarse a pines digitales con capacidad de interrupción para no perder ningún evento:

- **Sensor 1:** Se conecta al pin **GPIO 5** (D1 en la mayoría de las placas NodeMCU).
- **Sensor 2:** Se conecta al pin **GPIO 4** (D2 en la mayoría de las placas NodeMCU).
- **Alimentación:** Ambos sensores deben compartir una línea común de **GND** con el microcontrolador y recibir el voltaje adecuado (5V según el hardware del sensor).

PRECAUCION: los detectores CAJOE utilizan 5V para su funcionamiento, pero requieren una corriente del orden de los 100mA para operar. En cambio, el microcontrolador NodeMCU-ESP8266 opera con 5V y entrega 3.3V de salida, pero no proporciona la corriente necesaria para el funcionamiento de los detectores. Es por ello por lo que se debe procurar energizar cada sensor de forma independiente, sin utilizar las salidas de energía del microcontrolador.

2. Pantalla OLED SSD1306 (Protocolo I2C)

La visualización de datos en tiempo real (conteo y tasa CPM) se realiza mediante una pantalla OLED conectada por el bus I2C:

- **SDA (Datos):** Se conecta al pin **GPIO 14** (D5). No es necesario cablearla, pues está integrada al microcontrolador.
- **SCL (Reloj):** Se conecta al pin **GPIO 12** (D6). No es necesario cablearla, pues está integrada al microcontrolador.
- **VCC/GND:** Conectados a los pines de alimentación de 3.3V y tierra del ESP8266 respectivamente.

3. Esquema de Alimentación General

Dado que el ESP8266 gestiona la conexión WiFi y el envío de datos a Google Sheets, el consumo de corriente puede tener picos. Se recomienda:

- Alimentar el microcontrolador mediante el puerto **Micro-USB** o una fuente regulada de 5V al pin Vin.
- Hay que asegurar que la fuente de poder tenga capacidad suficiente para alimentar tanto al ESP8266 como a los sensores y la pantalla simultáneamente.

IMPORTANTE

- ESP8266 alimentado por USB 5V o Vin 5V.
- Sensores GM alimentados por fuente 5V independiente.
 - GND común entre todos los dispositivos.

Programación del microcontrolador

Instalar desde *Arduino IDE* → *Sketch* → *Include Library* → *Manage Libraries*:

- **ESP8266WiFi**
- **WiFiManager**
- **ESP8266HTTPClient**
- **WiFiClientSecure**
- **SSD1306Wire (ESP8266 OLED Driver)**

Se recomienda:

- Reemplazar la línea: `String webhookURL = "COLOCA AQUI URL DE GOOGLE SHEETS";`



- Ajustar la ventana de coincidencia si se desea experimentar: const unsigned long ventana = 50;

Código Fuente para el microcontrolador NodeMCU-ESP8266

```
/* Muon detector by LabTec v4.6 WiFi
 * By Ynerpoe
 * 26/04/2026
 */

#include <Wire.h>
#include "SSD1306Wire.h"
#include <ESP8266WiFi.h>
#include <WiFiManager.h>
#include <time.h>
#include <ESP8266HTTPClient.h>
#include <WiFiClientSecure.h>

// ----- PINES -----
#define SDAPin 14
#define SCLpin 12
#define Sens_pin1 5
#define Sens_pin2 4

// ----- DISPLAY -----
SSD1306Wire display(0x3c, SDAPin, SCLpin);

// ----- VARIABLES GLOBALES -----
volatile unsigned long coincidencias = 0;
volatile unsigned long t1 = 0;
volatile unsigned long t2 = 0;
const unsigned long ventana = 50; //Ventana para el ajuste de detección

unsigned long tiempo_inicio;
unsigned long ultimoEnvio = 0;

String webhookURL = "COLOCA AQUI URL DE GOOGLE SHEETS";

// Control de envío no bloqueante
int indiceEnvio = 0;
bool enviandoBuffer = false;

// ----- BUFFER DE EVENTOS -----
struct Evento {
    unsigned long cuentas;
    float tiempo_s;
    float cpm;
};

const int BUFFER_MAX = 500;
Evento bufferEventos[BUFFER_MAX];
int bufferCount = 0;

void bufferGuardarEvento(unsigned long cuentas, float tiempo_s, float cpm) {
    if (bufferCount < BUFFER_MAX) {
        bufferEventos[bufferCount++] = {cuentas, tiempo_s, cpm};
    }
}
```



```
}

// ----- INTERRUPTOS -----
void ICACHE_RAM_ATTR pulso1() {
  t1 = micros();
  if (t1 - t2 <= ventana) {
    coincidencias++;
    t1 = 0; t2 = 0;
  }
}

void ICACHE_RAM_ATTR pulso2() {
  t2 = micros();
  if (t2 - t1 <= ventana) {
    coincidencias++;
    t1 = 0; t2 = 0;
  }
}

// ----- FUNCIONES AUXILIARES -----
void actualizarDisplay(unsigned long c, float cpm) {
  display.clear();
  display.setTextAlignment(TEXT_ALIGN_CENTER);
  display.setFont(ArialMT_Plain_10);
  display.drawString(64, 0, "LABTEC MUON WiFi");
  display.drawHorizontalLine(0, 14, 128);
  display.setFont(ArialMT_Plain_16);
  display.drawString(64, 25, "LOGS: " + String(c));
  display.setFont(ArialMT_Plain_10);
  display.drawString(64, 50, "TASA: " + String(cpm, 2) + " cpm");
  display.display();
}

bool enviarGoogleSheets(unsigned long c, float segundos, float cpm) {
  if (WiFi.status() != WL_CONNECTED) return false;
  WiFiClientSecure client;
  client.setInsecure();
  HTTPClient http;
  if (!http.begin(client, webhookURL)) return false;
  http.addHeader("Content-Type", "application/json");
  String json = "{\"cuentas\":\"" + String(c) + "\",\"tiempo_s\":\"" + String(segundos, 3) + "\",\"cpm\":\"" + String(cpm, 2) + "\"}";
  int code = http.POST(json);
  http.end();
  return (code > 0);
}

void iniciarNTP() {
  configTime(-3 * 3600, 0, "pool.ntp.org", "time.nist.gov");
  time_t now;
  do { delay(500); time(&now); } while (now < 100000);
}

// ----- SETUP -----
void setup() {
  Serial.begin(115200);
}
```



```
display.init();
display.flipScreenVertically();
actualizarDisplay(0, 0);

pinMode(Sens_pin1, INPUT);
pinMode(Sens_pin2, INPUT);
attachInterrupt(digitalPinToInterrupt(Sens_pin1), pulso1, FALLING);
attachInterrupt(digitalPinToInterrupt(Sens_pin2), pulso2, FALLING);

tiempo_inicio = millis();

WiFiManager wm;
if (!wm.autoConnect("MuonDetector-Setup")) ESP.restart();

iniciarNTP();
}

// ----- LOOP OPTIMIZADO -----
void loop() {
  unsigned long c;
  noInterrupts();
  c = coincidencias;
  interrupts();

  float segundos = (millis() - tiempo_inicio) / 1000.0;
  float minutos = segundos / 60.0;
  float cpm = (minutos > 0.01) ? (c / minutos) : 0.0;

  static unsigned long ultimaCuenta = 0;
  if (c > ultimaCuenta) {
    bufferGuardarEvento(c, segundos, cpm);
    ultimaCuenta = c;
  }

  // Lógica de envío progresivo
  if (millis() - ultimoEnvio >= 60000 && !enviandoBuffer) {

    display.clear();
    display.setTextAlignment(TEXT_ALIGN_CENTER);
    display.setFont(ArialMT_Plain_10);
    display.drawString(64, 0, "LABTEC MUON WiFi");
    display.drawHorizontalLine(0, 14, 128);
    display.drawString(64, 30, "...enviando datos...");
    display.display();

    if (bufferCount > 0) {
      enviandoBuffer = true;
      indiceEnvio = 0;
    }
    ultimoEnvio = millis();
  }

  if (enviandoBuffer) {
    if (enviarGoogleSheets(bufferEventos[indiceEnvio].cuentas,
                          bufferEventos[indiceEnvio].tiempo_s,
```

```
        bufferEventos[indiceEnvio].cpm)) {  
    indiceEnvio++;  
}  
if (indiceEnvio >= bufferCount) {  
    bufferCount = 0;  
    enviandoBuffer = false;  
}  
} else {  
    actualizarDisplay(c, cpm);  
}  
}
```

Configuración de Google Sheets

1. Crear la hoja

- ✓ Crear una nueva hoja.
- ✓ Registrar el ID de la hoja (entre /d/ y /edit).
- ✓ Crear encabezados:
 - a. fecha
 - b. cuentas
 - c. tiempo_s
 - d. cpm

2. Crear el Script de Google

- ✓ El documento incluye el script completo.
- ✓ Citar fragmento clave: "sheet.appendRow([new Date(), data.cuentas, data.tiempo_s, data.cpm]);"

3. Publicar como aplicación web

- ✓ Extensiones → Apps Script
- ✓ Deploy → Nueva implementación
- ✓ Tipo: Aplicación web
- ✓ Acceso: Cualquiera
- ✓ Copiar la URL generada
- ✓ Pegarla en el código del ESP8266

RECUERDA: dentro de tu planilla, ve a **Extensiones => Apps Script** y pega el siguiente código:

Script de Google Sheets
<pre># Google Sheets script para recoger datos del detector # by Ynerpoe # abril 2026 function doPost(e) { try { if (!e !e.postData !e.postData.contents) { return ContentService.createTextOutput("No POST data received"); } var data = JSON.parse(e.postData.contents); var ss = SpreadsheetApp.getActiveSpreadsheet();</pre>

```
var sheet = ss.getSheetByName("Hoja 1");

sheet.appendRow([
  new Date(),
  data.cuentas,
  data.tiempo_s,
  data.cpm
]);

return ContentService.createTextOutput("OK");
}
catch (err) {
  return ContentService.createTextOutput("Error: " + err);
}
}

function doGet(e) {
  return ContentService.createTextOutput("Google Sheets Webhook activo");
}
```

4. Implementar como Aplicación Web

Este es el paso crucial para que internet pueda "ver" tu script:

- Haz clic en el botón **Implementar** (Deploy) => **Nueva implementación**.
- Selecciona tipo: **Aplicación web**.
- En "Quién tiene acceso", selecciona **Cualquiera** (esto permite que dispositivos externos envíen datos sin loguearse en Google).
- Copia la **URL de la aplicación web** generada.

5. Enviar los datos

Ahora puedes enviar datos desde cualquier lugar usando una petición **POST**. Por ejemplo, usando el código fuente en el microcontrolador del detector.

Nota de seguridad: Al configurar el acceso a "Cualquiera", la URL actúa como una llave. No la compartas públicamente, ya que cualquier persona con el enlace podría escribir en tu planilla. Si necesitas más seguridad, podrías añadir un campo de "token" o contraseña dentro del JSON que validas en el script antes de guardar los datos.

Pruebas iniciales

- ✓ **Verificar sensores**
 - Conectar solo un sensor y abrir el monitor serial.
 - Deberían aparecer pulsos aleatorios (~20–60 CPM).
- ✓ **Verificar interrupciones**
 - Tocar suavemente el tubo (sin golpear).
 - Debe aumentar la tasa de pulsos.
- ✓ **Verificar coincidencias**
 - Colocar ambos sensores alineados verticalmente.
 - Coincidencias típicas: **0.5–2 CPM**.

- ✓ **Verificar pantalla**
- ✓ Debe mostrar:
 - LOGS: N
 - TASA: X.X cpm
- ✓ **Verificar envío a Sheets**
 - Cada minuto el ESP8266 envía el buffer.
 - Deben aparecer filas nuevas en la hoja.

Solución de problemas

Problema	Causa probable	Solución
Pantalla en blanco	Dirección I2C incorrecta	Probar 0x3C o 0x3D
No detecta coincidencias	GND no común	Unificar tierras
No envía datos	Webhook incorrecto	Revisar URL y permisos
ESP8266 se reinicia	Fuente insuficiente	Usar 5V 1A mínimo
CPM muy alto	Ruido eléctrico	Separar cables de alimentación

Recomendaciones didácticas

- Explicar qué es un muón y su origen atmosférico.
- Comparar tasas medidas con valores típicos.
- Realizar experimentos:
 - Atenuación con materiales
 - Dependencia con la orientación
 - Variación con la altitud

Espero que todo funcione.